DeliverMe App Logic Flows

Generated from flows.md
DeliverMe workspace

Visual flowchart distilled from **flows.md**. The map shows how session restore, account flows, ride request, driver availability, Redis/BullMQ, and Socket.IO fit together.

Shared app · Client · Driver · Backend / worker · Gap / limitation

Flow 1: App Launch

Read AsyncStorage token and user type. Decode JWT locally; clear storage if invalid.

→

Route By Role

Valid client token goes to ClientHome. Valid driver token goes to DriverHome. Otherwise user starts at Home.

→

Account Entry

Client and driver can sign up, verify email code, or sign in to receive a role-scoped JWT.

→

Authenticated App

REST calls use Authorization: Bearer JWT. Socket connections send the JWT in handshake auth.

Client Journey

Flows 2-4: Account

Signup / verify / signin store userToken, userType=client, and userData.

↓

Flow 8: Choose Route

GPS + map picker select pickup and destination. Route distance and fare are prepared locally.

↓

Flow 9: Request Ride

POST /api/auth/client/request-ride creates a pending ride, caches it in Redis, and queues matching.

↓

Flow 10: Search Nearby Drivers

GET /api/rides/:rideId/nearby-drivers returns alive Redis drivers near pickup.

Driver Journey

Flows 5-7: Account

Signup uploads required documents, verifies email code, then stores userToken, userType=driver, and userData.

↓

Flow 11: Go Online

PATCH /driver/availability true updates MongoDB and queues driver-online work.

↓

Flow 12: Location + Heartbeat

Background task sends driverLocation when moved enough; otherwise refreshes heartbeat presence.

↓

Flow 13: Receive Ride Requests

DriverHome listens for ride_request and nearby-rides socket events.

Client creates pending ride

MongoDB ride + Redis geo/cache + BullMQ matchRide job.

Ride Matching

Driver becomes discoverable

MongoDB availability + Redis driver geo/presence + Socket.IO driver room.

Page 1 - High-level map

Flows 1-7 and 15

Account, Verification, And Session Flow

Both roles follow the same basic shape: validate input, create or find account, email a verification code, issue JWT after verification/signin, then route into the role home screen.

Client Account

Flow 2: Client Signup

Validate email, password, mobile. POST /api/auth/client/signup. Backend hashes password and sends verification code.

Flow 3: Client Verification

POST /api/auth/client/verify. Matching code sets clientVerified=true and returns JWT + profile.

Flow 4: Client Signin

POST /api/auth/client/signin. If verified, backend returns JWT. If not verified, UI asks for code.

ClientHome Session

Frontend stores userToken, userType=client, and userData, then resets navigation to ClientHome.

Driver Account

Flow 5: Driver Signup

Validate text fields and documents. POST multipart /api/auth/driver/signup. Backend uploads docs, hashes password, sends code.

Flow 6: Driver Verification

POST /api/auth/driver/verify. Matching code sets driverVerified=true and returns JWT + profile.

Flow 7: Driver Signin

POST /api/auth/driver/signin. If unverified, backend sends code again and UI asks for verification.

DriverHome Session

Frontend stores userToken, userType=driver, and userData, then resets navigation to DriverHome.

Session Restore

On launch, Home/signin/signup screens decode stored JWT locally. Expired or invalid token clears AsyncStorage.

Session Expired Event

AppNavigator listens for SESSION_EXPIRED and resets navigation back to Home.

Logout

No backend logout endpoint. Client clears storage and closes socket. Driver also goes offline when currently available.

Client Ride Request And Matching

Flows 8-10 and 13

This page shows the main business path: a client chooses a route, submits a ride, the backend records and caches it, and the worker pushes matching requests to driver socket rooms.

Flow 8: Pickup + Destination
ClientHome gets GPS, reverse-geocodes pickup, MapPicker selects pickup/destination, route distance is calculated.

→

Fare + Distance Guard
Frontend requires pickup, destination, fare, and routeDistance. Fare must meet the distance-based minimum.

→

Flow 9: Request Ride
POST /api/auth/client/request-ride with client JWT and route payload.

→

Backend Validation
JWT supplies client identity. Backend validates fare, coordinates, and haversine distance tolerance.

MongoDB Ride
Insert status=pending ride with GeoJSON pickup/destination, client profile, fare, routeDistance, and expiresAt.

→

Redis Ride Cache
addRideToGeo() stores ride location and cached ride details for matching/search.

→

BullMQ Job
rideQueue.add("matchRide", { rideId }) responds immediately; worker runs matching later.

→

Socket Broadcast
Matching worker emits ride_request to nearby driver:{driverId} rooms.

Client Search Screen

Navigate To SearchingDriver
After successful ride creation, frontend passes rideId to SearchingDriver.

↓

Flow 10: Nearby Drivers
GET /api/rides/:rideId/nearby-drivers?radiusKm=5&limit=20 uses pickup location to query Redis.

↓

Display Results
Shows driver name, mobile, vehicle, and distance. If empty, shows no available drivers yet.

Driver Request Screen

Flow 13: Socket Events
DriverHome listens for ride_request and nearby-rides.

↓

Request List
nearby-rides replaces list; ride_request appends and dedupes.

↓

Current Limitation
Accept Ride button exists, but no active accept-ride API call or socket event was found.

Driver Runtime And Availability

Driver online state is maintained through REST, background location updates, Socket.IO, Redis geo keys, and worker jobs.

Going Online

Toggle Online

DriverHome asks location permission, gets current GPS, initializes socket, and attaches ride listeners.

PATCH Availability True

PATCH /api/auth/driver/availability with location updates MongoDB isAvailable=true and GeoJSON location.

Queue driver-online

Backend adds driver-availability job with driver profile, vehicle, location, and timestamp.

Worker Side Effects

Adds driver to Redis, finds nearby pending rides, publishes driver:rides, emits nearby-rides to driver room.

Staying Online

Background Task

Expo background location task checks AsyncStorage driverAvailable before sending updates.

driverLocation

Sent on first location or when movement is at least 2000 meters.

driverHeartbeat

Sent when there is no location data or movement is below threshold.

Redis Presence

Backend refreshes drivers:geo, driver:{id}:alive, driver:{id}:online, and driver:{id}:socket keys.

Socket Registration

Driver emits driver:register, driverOnline, and driver:availability after going online.

Ride Notifications

Driver receives nearby-rides when going online and ride_request when a new pending ride matches.

Going Offline

PATCH availability false, emit driver:availability false, stop tracking, close socket, clear request list.

Offline cleanup happens in both REST worker logic and socket handler logic: Redis driver geo and presence keys are removed, and MongoDB isAvailable is set to false.

Backend Support Flows And API Surface

Flows 16-17 and endpoint summary

These supporting pieces keep ride data searchable, expire stale rides, and expose development/debug visibility.

MongoDB

Stores clients, drivers, and rides. Ride documents include GeoJSON pickup/destination, status, fare, client details, and expiration.

Redis Geo + Cache

Stores drivers and rides for fast nearby lookups. Driver presence TTLs determine who is alive/online.

BullMQ Workers

driver-availability and ride matching jobs decouple frontend response time from background matching work.

Ride Expiration Worker

Runs Every 10 Seconds

Finds up to 50 pending rides whose expiresAt is in the past.

Expire Ride

Updates status to expired and sets expiredAt.

Redis Cleanup

Removes expired ride from Redis geo.

Limitation

rideExpired socket emits are present but commented out, so frontend is not notified directly.

Redis Debug Routes

Driver Visibility

Inspect drivers:geo, cached driver data, one driver location, alive TTL, and full driver geo data.

Nearby Testing

GET /api/debug/redis/nearby?lat=...&lng=...&radius=5 finds drivers near a coordinate.

Ride Visibility

Inspect rides:geo, cached ride data strings, and combined full ride state.

Development Only

No active frontend caller was found for these debug endpoints.

Area	Endpoint/Event	Main Writer	Main Reader
Auth	/api/auth/client/* and /api/auth/driver/*	Frontend screens	Auth controllers + MongoDB
Ride	POST /client/request-ride	ClientHome	MongoDB, Redis, rideQueue
Driver	PATCH /driver/availability	DriverHome	MongoDB, driverQueue, Redis

Socket	ride_request, nearby-rides, driverLocation, driverHeartbeat	Worker / driver app	DriverHome / Redis presence
Debug	/api/debug/redis/*	Backend route	Developer inspection

Page 5 - Backend support